Reviewer Pack — Minimal Rank of Quasigroup Representations vs Tseitin Formul…

Entry 33f8ea7c6adc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-24 11:18:03 UTC

Minimal Rank of Quasigroup Representations vs Tseitin Formula Resolution Depth

Entry ID: `33f8ea7c6adc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-24 11:18:03 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Algebra (Quasigroup Theory) **Field B** (complexity object): Complexity Theory: Tseitin Formula Resolution Depth

Statement:

> ['For every Tseitin formula F over n variables, there exists a quasigroup representation Q of size 2^n such that the Resolution refutation tree of F has depth ≥ 2^Ω(rank(Q)).', 'Equivalently, for any Tseitin formula F, the resolution refutation depth is lower bounded by a function of the minimal rank of a quasigroup that can represent its clauses.', 'The ratio of the resolution refutation depth to the minimal rank of the quasigroup is exponential in the size of the formula.']

Rationale (proposer's reasoning):

> ['Quasi-groups are algebraic structures that generalize groups and may provide a rich source of invariants for complexity theory. If quasigroup representations can be used to bound resolution refutation depth, it could shed light on the complexity of Tseitin formulas.', 'The relationship between quasigroups and logical structures has not been extensively explored in complexity theory, making this a potential area for novel insights.', 'A direct connection between quasigroup representations and Tseitin formula resolution depth would offer a new approach to understanding the hardness of Tseitin problems.']

Taxonomy category: `TSEITIN_RES` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `9be8f4300eb79908`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if for at least 80% of generated Tseitin formulas with n variables (n ≤ 40), the ratio of resolution refutation depth to minimal quasigroup rank exceeds an exponential threshold based on the formula size, specifically |refutation_depth - min_rank| > e^(0.1 * log2(n)).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	UNCERTAIN	1.00	HITS	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=1.6s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
        for i in range(1, n+1):
            clause = [f'-{variables[i-1]}', f'{variables[n+i-1]}']
            clauses.append(clause)
            clause = [f'{variables[i-1]}', f'-{variables[n+i-1]}']
            clauses.append(clause)
        for i in range(n+1, 2*n):
            for j in range(i+1, 2*n):
                clause = [f'-{variables[i-1]}', f'-{variables[j-1]}', f'{variables[2*n+i-j-1]}']
                clauses.append(clause)
        return variables, clauses

    def is_quasigroup(q):
        n = len(q)
        for i in range(n):
            for j in range(n):
                if q[i][j] < 0 or q[i][j] >= n:
                    return False
                for k in range(n):
                    if q[q[i][j]][k] != q[i][q[j][k]]:
                        return False
        return True

    def resolution_refutation_depth(q, clauses):
        n = len(q)
```

```
    visited = set()
    stack = []
    for clause in clauses:
        stack.append(clause)
    while stack:
        clause = stack.pop()
        if all(x not in visited for x in clause):
            visited.update(clause)
            for i in range(n):
                for j in range(n):
                    if q[i][j] == -1:
                        continue
                    new_clause = [f'-{q[i][j]}']
                    if any(c in new_clause for c in clause):
                        continue
                    stack.append(new_clause)
    return len(visited) + 1

def min_quasigroup_rank(clauses):
    n = int(math.sqrt(len(clauses)))
    q = [[-1] * n for _ in range(n)]
    rank = 0
    for i, clause in enumerate(clauses):
        if all(x not in q[i] for x in clause):
            rank += 1
            for x in clause:
                q[i][x] = i
    return rank

variables, clauses = generate_tseitin_formula(5)
quasigroups = []
for _ in range(30):
    q = [[random.randint(0, n-1) for _ in range(n)] for _ in range(n)]
    if is_quasigroup(q):
        quasigroups.append((q, min_quasigroup_rank(clauses)))

refutation_depths = [resolution_refutation_depth(q, clauses) for q, rank in quasigroups]
ranks = [rank for q, rank in quasigroups]

if not quasigroups:
    return {
        "metric_name": "refutation_depth_to_rank_ratio",
        "metric_value": 0,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

refutation_depth_mean = sum(refutation_depths) / len(refutation_depths)
rank_mean = sum(ranks) / len(ranks)
ratio_mean = refutation_depth_mean / rank_mean

return {
    "metric_name": "refutation_depth_to_rank_ratio",
    "metric_value": ratio_mean,
    "instances_tested": len(quasigroups),
```

```
        "conjecture_holds": ratio_mean > math.exp(0.1 * math.log2(len(clauses))),
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 37))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    refutation_depth_mean = sum(r["metric_value"] for r in results) / len(results)
    rank_mean = sum(1/r["instances_tested"] * r["metric_value"] for r in results) / sum(1/r["insta
    support_fraction = sum(r["conjecture_holds"] for r in results) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={refutation_depth_mean} std=0 support_fraction={support_fra
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={refutation_depth_mean} std=0 support_fraction={support_fra
    else:
        first_failing_seed = next(i for i, r in enumerate(results) if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"not enough quasigroups\" first_failing_seed={se
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_983afc29.py", line 114, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_983afc29.py", line 77, in run_trial
    variables, clauses = generate_tseitin_formula(5)
                         ^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_983afc29.py", line 25, in generate_tsei
    clause = [f'-{variables[i-1]}', f'{variables[n+i-1]}']
                                    ~~~~~~~~~^^^^^^^^
IndexError: list index out of range
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed before producing any data, which means that it was unable to complete its intended task of verifying the conjecture. | next: Re-run the test with a different seed or investigate the cause of the crash to ensure that the test can be completed successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13183
2	preregistration	ollama_remote	glm4:latest	0	0	5976
3	novelty	ollama_remote	glm4:latest	0	0	4782
4	novelty	ollama_remote	glm4:latest	0	0	5175
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45622
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14115
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31999
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	26236
9	judge	ollama_remote	glm4:latest	0	0	51765

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 198853 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/33f8ea7c6adc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/33f8ea7c6adc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/33f8ea7c6adc.tar.gz` (if generated)